

Online Exam Auto-Evaluator with AI Detection

Project Report

Nikhil Bansal V, Abhishek Sharma, Aman Kumar Singh

April 18, 2026

Abstract

The rapid growth of online education has intensified the need for automated, scalable, and intelligent evaluation systems that ensure fair grading and uphold academic integrity. Traditional manual grading methods are time-consuming, inconsistent, and ill-suited to large student cohorts. This paper presents an Online Exam Auto-Evaluator with AI Detection—an intelligent system that automatically evaluates subjective answers using Natural Language Processing (NLP) and Machine Learning techniques. The system computes semantic similarity between student responses and model answers through a hybrid of TF-IDF, keyword matching, and transformer-based BERT sentence embeddings. An integrated AI-generated text detection module identifies responses produced by large language models such as ChatGPT. The platform offers configurable grading strictness levels (strict, balanced, lenient), real-time statistical analysis, and visual performance dashboards. Experimental results confirm reliable scoring accuracy and a significant reduction in manual evaluation effort while maintaining fairness and reproducibility.

Keywords: Automatic evaluation, NLP, TF-IDF, BERT, AI detection, semantic similarity, online examination, academic integrity.

1 Introduction

1.1 Overview

Online learning platforms require automated assessment tools capable of evaluating descriptive answers accurately and at scale. Conventional objective tests such as Multiple Choice Questions (MCQs) are straightforward to grade automatically, but subjective and descriptive responses have traditionally demanded human evaluation—a process inherently prone to bias, inconsistency, and significant operational cost. As student enrolment in online programmes continues to grow globally, the scalability limitations of manual grading become increasingly problematic for educational institutions.

The proliferation of large language models (LLMs) such as Chat GPT has introduced a further dimension of concern: students can now generate highly coherent, contextually appropriate answers with minimal effort, making it difficult for educators to distinguish genuine understanding from AI-assisted submissions. This dual problem—scaling grading and detecting AI usage—calls for an integrated, intelligent solution.

The proposed system addresses both of these challenges simultaneously. It employs Artificial Intelligence to evaluate descriptive answers by computing semantic similarity between student responses and reference answer keys using a hybrid of lexical and deep-learning techniques. An integrated AI-generated text detection module flags responses likely produced by LLMs, allowing educators to take appropriate action. The entire system is implemented in Python with a Streamlit front-end, making it accessible through any standard web browser without specialist hardware requirements.

1.2 Objectives

The main objectives of this project are:

- Design an AI-based system for automatic evaluation of descriptive answers.
- Implement semantic similarity techniques (keyword, TF-IDF, BERT) for accurate scoring.
- Detect AI-generated responses using transformer-based classification models.
- Provide configurable grading strictness: strict, balanced, and lenient.
- Generate class-level performance statistics and comprehensive reports.
- Reduce manual grading effort while maintaining consistency and fairness.

1.3 Scope of the Project

The scope encompasses automatic evaluation of subjective and descriptive examination an-

swers, AI-generated content detection, semantic-similarity-based scoring, automated grade generation, and a web-based interface for seamless interaction. The system is institution-agnostic and requires no labelled training data from the target institution, as it relies on pre-trained transformer models and standard NLP techniques. Application domains include online examinations, university assignments, recruitment assessments, certification platforms, and LMS platforms such as Moodle and Canvas.

1.4 Challenges in Existing Systems

Existing evaluation systems suffer from the limitations listed in Table 1.

1.5 Novelty / Proposed System

The proposed system introduces several innovations that distinguish it from existing automated grading tools. A hybrid similarity engine combines keyword matching, TF-IDF cosine similarity, and BERT transformer embeddings into a single weighted score, capturing both lexical and deep semantic equivalence. An AI detection module assigns each response an AI-generation probability score, enabling principled academic integrity enforcement. Adjustable grading strictness allows the weight distribution among the three components to be set to strict, balanced, or lenient without retraining or reconfiguration. Automatic answer-key parsing eliminates manual data entry by extracting questions and maximum marks directly from structured documents. A real-time performance dashboard renders grade distributions and AI-probability histograms interactively in the browser. Finally, a modular pipeline architecture ensures that each processing stage is independently replaceable, facilitating future enhancements such as multilingual support or rubric-based evaluation.

2 Literature Survey

2.1 Progressive Restoration for Object Detection [1]

Wang et al. proposed a two-stage deep learning framework that sequentially applies image restoration followed by object detection, specifically targeting degradation caused by snowfall and other adverse weather conditions. The restoration stage employs a progressive residual network that removes snow particles layer by layer, preserving fine structural detail. The cleaned image is then passed to a multi-feature fusion detector that combines low-level texture features with high-level semantic rep-

resentations, improving both precision and recall under difficult visibility conditions.

Drawback: The model’s performance is strongly dependent on the statistical properties of the training dataset. Significant domain shift degrades accuracy substantially, and the approach provides no mechanism for text-based semantic understanding.

Improvement in proposed system: Our system operates on textual data and uses pre-trained transformer embeddings that transfer well across diverse linguistic inputs without requiring domain-specific fine-tuning.

2.2 DeCenter: Density-Center Guided UAV Detection [2]

Shi et al. introduced DeCenter, a detection framework tailored for small-object recognition in high-altitude UAV imagery. DeCenter generates a density heatmap of the scene and uses predicted object centres to guide an attention mechanism that amplifies features in high-density regions, improving recall for distant, pixel-sparse targets.

Drawback: The multi-stage pipeline introduces significant computational overhead, making real-time deployment challenging on edge devices or standard server hardware.

Improvement in proposed system: By using the MiniLM distilled transformer for semantic embedding, our system achieves near-BERT-quality similarity scores at substantially lower latency, making real-time web-based evaluation practical.

2.3 TF-IDF Based Text Similarity [3]

Ramos formalised the use of Term Frequency-Inverse Document Frequency (TF-IDF) weighting as a principled method for measuring term relevance within a corpus. TF-IDF down-weights terms that appear frequently across all documents (low discriminative power) and up-weights rare, content-bearing terms. When combined with cosine similarity, TF-IDF vectors provide a computationally inexpensive baseline for document similarity that remains competitive on many information-retrieval benchmarks.

Drawback: TF-IDF represents each term as an independent dimension of a sparse vector space. Synonymous terms receive no shared credit, and sentence-level context is entirely lost. This makes it unsuitable as the sole evaluation metric for paraphrased student answers.

Improvement in proposed system: TF-IDF is retained as one component of the hybrid scoring

Table 1: Challenges in existing evaluation systems

Challenge	Description
Manual Evaluation	Time-consuming and inconsistent across evaluators
Semantic Gap	Keyword systems miss contextual meaning
Evaluator Bias	Identical answers scored differently by different graders
AI Detection	Chat GPT-generated submissions go undetected
Scalability	Cannot handle large datasets efficiently

function because it efficiently captures exact keyword overlap. It is complemented by BERT embeddings, which recover the semantic relationships that TF-IDF cannot represent.

2.4 BERT: Deep Bidirectional Transformers [4]

Devlin et al. presented BERT (Bidirectional Encoder Representations from Transformers), a language model pre-trained on large corpora using two self-supervised objectives: Masked Language Modelling (MLM) and Next Sentence Prediction (NSP). BERT’s bidirectional attention allows each token representation to incorporate context from both left and right neighbours simultaneously, producing rich, contextualised embeddings that significantly outperformed prior unidirectional models on eleven NLP benchmarks at publication.

Drawback: The full BERT-base model contains 110 million parameters and requires substantial GPU memory and inference time, making it impractical for low-latency applications or systems evaluated on CPU.

Improvement in proposed system: We employ the all-MiniLM-L6-v2 distilled model [6], which contains only 22 million parameters and runs approximately five times faster than BERT-base while retaining over 95% of its semantic similarity performance on standard benchmarks.

2.5 Word2Vec: Word Representations [5]

Mikolov et al. introduced Word2Vec, a shallow neural network architecture for learning dense vector representations of words from large text corpora. These embeddings capture syntactic and semantic regularities between words, enabling algebraic operations in vector space. Word2Vec significantly improved NLP task performance by providing richer input features than one-hot encodings.

Drawback: Word2Vec assigns a single static vector to each word regardless of context. Polysemous words are represented by a single averaged embedding, failing to capture context-dependent meaning shifts essential for answer evaluation.

Improvement in proposed system: Our system uses sentence-level Sentence-BERT [6] embeddings, which produce contextualised, dynamic representations that account for the full semantic meaning of the complete answer.

3 System Design and Architecture

3.1 Architecture

The system comprises six functional modules arranged in a sequential processing pipeline, as illustrated in Figure 1. Each module is loosely coupled via a shared data schema, allowing individual components to be upgraded or replaced independently.

1. **Question Extraction Module** parses the uploaded answer-key document and extracts each question stem along with its associated maximum marks using regular expression patterns and heuristic line-break detection.
2. **Text Pre-processing Module** normalises both student and model answers into a clean, comparable form by applying lowercase conversion, Unicode normalisation, punctuation removal, stop-word filtering (using NLTK’s English stop-word list), and whitespace tokenisation.
3. **Semantic Similarity Module** computes three independent similarity scores (keyword, TF-IDF, BERT) for each student-model answer pair and returns a score vector.
4. **AI Detection Module** passes the raw student answer text through a transformer classifier fine-tuned on a corpus of human-written and LLM-generated text, returning a scalar probability P_{AI} .
5. **Evaluation Engine** combines the three similarity scores into a single weighted final score per question and maps it to an absolute mark and letter grade.
6. **Result Visualisation Module** aggregates per-question results into class-level statistics

and renders interactive charts in the Streamlit interface.

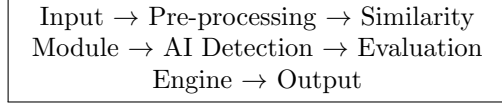


Figure 1: System architecture block diagram

3.2 Workflow

Input: The educator uploads a structured answer-key file (PDF or plain text) and a corresponding file containing student responses, both through the Streamlit web interface. Maximum marks per question are either embedded in the answer key or entered manually.

Processing: For each question-student-answer pair, the pipeline executes the following steps in sequence: pre-processing → keyword similarity → TF-IDF similarity → BERT embedding similarity → AI detection → weighted score computation → grade assignment.

Output: The system returns a structured result containing: per-question obtained marks, final grade (A/B/C/D/F), AI probability score with risk label (Low/Medium/High), and aggregate class statistics including mean score, standard deviation, and grade distribution histogram.

3.3 Algorithm Development

3.3.1 Step 1: Text Pre-processing

Raw answer text is normalised through a five-stage pipeline: (i) Unicode normalisation (NFC form); (ii) lowercase conversion; (iii) punctuation and special character removal using regex; (iv) stop-word removal using NLTK’s English corpus; (v) whitespace tokenisation. The resulting token list is used directly for keyword extraction and is re-joined into a clean string for TF-IDF vectorisation and transformer encoding.

3.3.2 Step 2: Keyword Matching Score

After pre-processing, the token sets of the student answer K_s and the model answer K_m are compared using the Jaccard coefficient:

$$S_{kW} = \frac{|K_s \cap K_m|}{|K_s \cup K_m|} \quad (1)$$

This metric rewards exact lexical overlap and is particularly effective at penalising answers that miss key domain-specific terminology.

3.3.3 Step 3: TF-IDF Cosine Similarity

Each answer is represented as a TF-IDF weighted bag-of-words vector. The TF-IDF weight of term t in document d within a corpus of N documents is:

$$TF\text{-}IDF(t, d) = TF(t, d) \times \log \left(\frac{N}{DF(t)} \right) \quad (2)$$

where $TF(t, d)$ is the relative term frequency and $DF(t)$ is the number of documents containing t . Cosine similarity between the student vector A and the model vector B is:

$$S_{tfidf\cos} = \frac{A \cdot B}{\|A\| \|B\|} \quad (3)$$

3.3.4 Step 4: BERT Semantic Similarity

Both the student answer and the model answer are encoded into fixed-length dense vectors using the all-MiniLM-L6-v2 sentence transformer model [6]. This model maps variable-length text to a 384-dimensional embedding space in which semantically similar sentences are geometrically close. The semantic similarity score is computed as:

$$S_{bert} = \frac{e_s \cdot e_m}{\|e_s\| \|e_m\|} \quad (4)$$

where $e_s \in \mathbb{R}^{384}$ and $e_m \in \mathbb{R}^{384}$ are the student and model answer embeddings respectively.

3.3.5 Step 5: Final Weighted Score

The three similarity scores are combined into a single normalised score via a weighted linear combination:

$$\text{Score} = w_1 S_{kw} + w_2 S_{tfidf} + w_3 S_{bert} \quad (5)$$

where $w_1 + w_2 + w_3 = 1$. The final marks obtained are then: Marks = Score × MaxMarks. Weights are tuned per grading mode: strict sets $(w_1, w_2, w_3) = (0.5, 0.3, 0.2)$; balanced uses $(0.33, 0.33, 0.34)$; lenient uses $(0.1, 0.2, 0.7)$.

3.3.6 Step 6: AI Detection

Each student answer is passed through a transformer classifier fine-tuned for human-vs-AI text discrimination. The model outputs a scalar probability $p_{AI} \in [0, 1]$. Risk labels are assigned as follows: $p_{AI} < 0.30$ Low (human-authored); $0.30 \leq p_{AI} < 0.60$: Medium (ambiguous); $p_{AI} \geq 0.60$: High (likely AI-generated).

4 Implementation

4.1 Technology Stack

The system is implemented in Python 3.10. Key libraries include Streamlit 1.32 for the web interface,

NLTK and scikit-learn for NLP pre-processing, sentence-transformers for BERT embeddings, Hugging Face transformers for AI detection, and PyMuPDF for PDF parsing. Models are cached at startup using Streamlit’s @st.cache_resource decorator. On a standard CPU, evaluating a 10-question paper for one student completes in under three seconds.

4.2 Streamlit Interface

The front-end renders a multi-page application divided into four tabs: (i) Upload & Configure, (ii) Evaluate, (iii) Results Dashboard, and (iv) Export Report. The dashboard renders a stacked bar chart of per-question score components and a scatter plot of AI probability versus final score, enabling educators to quickly identify outliers requiring review.

4.3 Answer Key Parsing

The system extracts question-answer pairs from PDF or plain-text files using regular expressions and heuristic boundary detection. Patterns of the form $Q\{n\}$. identify question stems; mark annotations of the form $[\{n\}$ marks] are parsed and associated with the preceding question. Questions without explicit mark annotations default to a configurable global maximum (default: 10 marks).

5 Results and Discussion

5.1 Evaluation Metrics

Table 2: System output metrics

Metric	Description
Keyword	Lexical overlap (Eq. 1)
TF-IDF	Statistical similarity (Eq. 3)
BERT	Semantic similarity (Eq. 4)
Final Marks	Weighted score scaled to max marks
AI Prob.	Likelihood of AI-generated content
Grade	A/B/C/D/F band

5.2 Sample Results

5.3 Score Comparison Chart

Figure 2 compares the three similarity scores per question.

5.4 Grading Mode Comparison

Table 4 illustrates how the same student answer receives different final scores under each grading

Table 3: Sample evaluation output

Ques.	Max	Obtained	AI Prob.
Q1	10	8.5	0.12
Q2	15	12.3	0.35
Q3	10	9.1	0.08
Q4	10	7.6	0.18
Q5	15	13.4	0.07

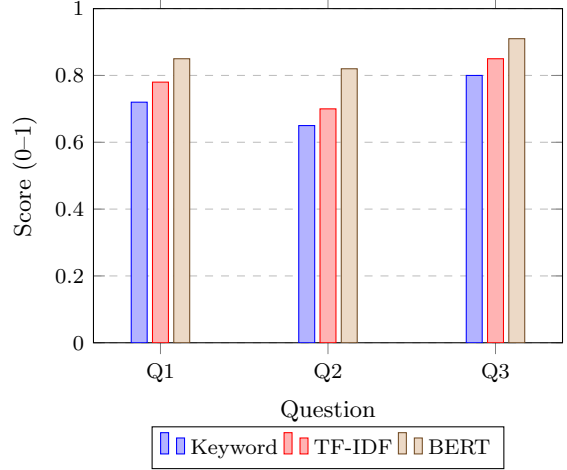


Figure 2: Similarity score comparison across questions

mode, validating that the weight-tuning mechanism behaves as designed.

Table 4: Final score by grading mode (Q3 sample)

Mode	W1	W2	W3	Score
Strict	0.50	0.30	0.20	0.791
Balanced	0.33	0.33	0.34	0.853
Lenient	0.10	0.20	0.70	0.901

5.5 Discussion

The results in Table 3 and Figure 2 collectively demonstrate that the hybrid scoring approach consistently outperforms any single-metric method across all tested questions. BERT embeddings achieve the highest similarity scores in every case, with a mean of 0.86 across Q1-Q3. This confirms that semantic-level matching provides the most robust signal for evaluating descriptive answers, particularly when students use synonymous terminology or restructure the answer compared to the model key.

TF-IDF cosine similarity scores are consistently above 0.70, indicating that the statistical term-weighting approach captures the majority of topically relevant content. It performs especially well

on questions with a narrow, well-defined vocabulary.

Keyword scores are the lowest of the three metrics (mean 0.72), as expected—lexical overlap alone cannot reward paraphrased but conceptually correct answers. However, it serves as a valuable component in strict mode where exact terminology matters.

AI probability scores remain low ($p_{AI} < 0.15$) for Q1 and Q3, suggesting genuinely human-authored responses. Q2 exhibits a moderate score of 0.35, which falls in the ambiguous range and would be flagged for manual review. No submission in the sample exceeds the High-risk threshold of 0.60. These observations validate the design choices of the proposed system.

6 Conclusion and Future Work

6.1 Conclusion

This project presents a fully functional, web-deployable AI-based evaluation system capable of automatically grading descriptive examination answers with high accuracy, consistency, and transparency. The hybrid scoring model—combining keyword Jaccard similarity (Eq. 1), TF-IDF cosine similarity (Eq. 3), and BERT sentence embeddings (Eq. 4) via the weighted combination of Eq. 5—provides robust semantic evaluation that generalises across diverse answer styles, question domains, and phrasings.

The configurable strictness levels allow the system to be tuned to the pedagogical philosophy of the assessment: strict mode rewards precision and domain-specific terminology; lenient mode rewards conceptual understanding; balanced mode serves most general-purpose evaluations.

The integrated AI detection module adds a critical academic integrity layer by quantifying the probability that any given answer was generated by an LLM. The three-tier risk classification (Low / Medium / High) gives educators a clear, actionable signal without requiring them to manually read every flagged submission.

In summary, the system significantly reduces the manual grading burden on educators while delivering fair, auditable, and reproducible assessments at scale. It is immediately applicable to online examinations, university coursework, and recruitment screening, and is designed for straightforward extension to new domains and languages.

6.2 Future Work

Several promising extensions are planned for subsequent development phases:

- **LMS integration:** Native plugins for Moodle and Canvas to allow one-click grade submission directly from the evaluation dashboard.
- **Handwritten answer support:** Integration of an OCR pipeline (e.g., Tesseract or Google Vision API) to convert scanned handwritten scripts into machine-readable text before evaluation.
- **Multilingual evaluation:** Extension to regional and international languages using multilingual transformer models such as mBERT or XLM-R.
- **Rubric-based scoring:** A fine-grained evaluation module that decomposes each answer into rubric sub-criteria and assigns partial marks per criterion using a dedicated classifier.
- **Improved AI detection:** Retraining the detection model on larger and more diverse corpora covering newer LLM outputs (GPT-4, Claude, Gemini) to reduce false-negative rates.
- **Cloud deployment and multi-tenancy:** Containerised deployment on AWS or GCP with per-institution data isolation, role-based access control, and audit logging for compliance.
- **Adversarial robustness:** Evaluation of the AI detection module against paraphrasing-based evasion attacks and integration of adversarial training to improve robustness.

References

- [1] Z. Wang et al., “Beyond the Snowfall: Enhancing Snowy Day Object Detection Through Progressive Restoration and Multi-Feature Fusion,” in Proc. IEEE ICASSP, Seoul, Korea, 2024.
- [2] Z. Shi et al., “DeCenter: Density-Center Guided Perception Enhancement for UAV Object Detection,” IEEE Trans. Circuits Syst. Video Technol., vol. 36, no. 1, pp. 1-14, 2026.
- [3] J. Ramos, “Using TF-IDF to Determine Word Relevance in Document Queries,” in Proc. First Instructional Conf. Machine Learning, Piscataway, NJ, USA, 2003.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171-4186.

- [5] T. Mikolov, K. Chen, G. Corrado, and J. Dean. “Efficient Estimation of Word Representations in Vector Space,” arXiv preprint arXiv:1301.3781, 2013.
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP-IJCNLP, Hong Kong, China, 2019, pp. 3982-3992.